Rangkumin Installation Guide for Beginners

Document version: 1.0 · September 7, 2026

This guide is written for people who are not familiar with terminals, databases, or Cloudflare. Follow every step in order and do not skip the **Expected result** sections.

> Never share tokens, private keys, user email addresses, receipt images, or real financial data through chat, screenshots, GitHub issues, or the repository.

1. Choose an installation method

Rangkumin can be run in two ways:

Option	Best for	Final result
A. Local computer	Learning, feature testing, and development	Available only from your computer
B. Cloudflare	Production use by a couple	Available online through a protected domain

If this is your first attempt, complete **Option A** first. Continue to **Option B** after the local application works.

2. Terms you should know

- **Terminal**: an application that runs text commands. Use PowerShell on Windows or Terminal on macOS.
- **Repository/source code**: the folder containing all Rangkumin code.
- **D1**: the Cloudflare database that stores production data.
- **Worker**: the backend application running on Cloudflare.
- **Cloudflare Access**: the login page that limits the application to two email addresses.
- **Secret**: confidential configuration that must never be committed to Git.
- **VAPID key**: a key pair used to send Web Push notifications.
- **Migration**: a file that creates or updates the database structure.

Option A: Install on a local computer

3. Install Node.js

1. Open https://nodejs.org/.
2. Download Node.js **22 LTS or newer**.
3. Run the installer.
4. Keep the default options until installation is complete.
5. Close and reopen PowerShell or Terminal.
6. Run:

```
node --version
npm --version
```

Expected result: the first command shows `v22` or higher, and the second command shows an npm version number.

If you see `node is not recognized` or `command not found`, restart the computer and check again.

4. Install Git

Windows

1. Open https://git-scm.com/download/win.
2. Run the Git installer.
3. Keep the default options.
4. Close and reopen PowerShell.

macOS

1. Open Terminal.
2. Run `git --version`.
3. If macOS asks to install Command Line Tools, select **Install**.

Verify Git:

```
git --version
```

Expected result: a Git version number is displayed.

5. Get the source code

Recommended method: Git

1. Choose a folder for the project.
2. Open PowerShell or Terminal in that folder.
3. Run:

```
git clone https://github.com/andikafadil28/rangkumin.git
cd rangkumin
```

Alternative without Git clone

1. Open https://github.com/andikafadil28/rangkumin.
2. Click **Code**.
3. Click **Download ZIP**.
4. Extract the ZIP file.
5. Open the extracted folder.
6. Right-click an empty area and select **Open in Terminal**, or open Terminal and navigate to that folder.

Expected result: the terminal is inside a folder containing `package.json`, `README.md`, and the `src` folder.

6. Install dependencies

Make sure the terminal is inside the Rangkumin folder, then run:

```
npm ci
```

Wait for it to finish. This process may take several minutes.

Expected result: the command finishes without `npm ERR!`.

Dependency warnings that do not stop the command usually do not block installation. Do not run `npm audit fix --force` without understanding its impact.

7. Create the local database

Run:

```
npm run db:migrate:local
```

If asked to continue the migration, answer `y` or `yes`.

Expected result: all migrations show a successful status.

8. Add dummy data

Run:

```
npm run db:seed:local
```

The dummy data creates two practice users:

- User One: `user1@example.invalid`
- User Two: `user2@example.invalid`

Expected result: the command finishes without a database error.

These are not real email accounts. Do not replace seed data with personal information.

9. Start the backend

The application configuration includes a remote Workers AI binding. On first use, create a Cloudflare account if needed, then sign in:

```
npx wrangler login
```

A browser opens for authorization. Return to the terminal after it succeeds. Signing in does not turn local data into production data, but Receipt Scan may still consume Workers AI quota when used.

In the first terminal, run:

```
npm run dev
```

Keep this terminal open. Do not press `Ctrl+C` while using the application.

Expected result: the terminal displays the Worker address, usually `http://localhost:8787`.

10. Start the frontend

1. Open a second terminal in the same Rangkumin folder.
2. Run:

```
npm run dev:frontend
```

3. Keep the second terminal open.

Expected result: the terminal displays `http://localhost:5173`.

11. Open the local application

1. Open Chrome, Edge, or another modern browser.
2. Enter http://localhost:5173.
3. Wait for the dashboard to appear.

Local mode automatically uses User One. This is only a development mechanism and must never be used for a public website.

Local checklist:

- The dashboard opens.
- A dummy transaction can be created.
- Savings, budgets, reminders, and settings pages open.
- Reloading the page does not produce a blank screen.

12. Stop the local application

1. Return to the frontend terminal.
2. Press `Ctrl+C`.
3. Return to the backend terminal.
4. Press `Ctrl+C`.

Local data remains under `.wrangler/` and will be available the next time the application starts.

13. Start the application again

You do not need to reinstall dependencies, migrations, or seed data every time.

1. Open the first terminal in the Rangkumin folder and run `npm run dev`.
2. Open a second terminal in the same folder and run `npm run dev:frontend`.
3. Open http://localhost:5173.

Option B: Install online on Cloudflare

14. Prepare Cloudflare requirements

Before starting, prepare:

- a Cloudflare account;
- a domain managed by Cloudflare DNS;
- two email addresses that will use Rangkumin;
- one contact email address for Web Push;
- a payment method if Cloudflare usage exceeds free quotas.

Workers AI for Receipt Scan may incur charges. Check current pricing at https://developers.cloudflare.com/workers-ai/platform/pricing/.

15. Sign in to Cloudflare from the terminal

From the Rangkumin folder, run:

```
npx wrangler login
```

1. A browser window opens.
2. Sign in to Cloudflare.
3. Select **Allow/Authorize**.
4. Return to the terminal.
5. Verify:

```
npx wrangler whoami
```

Expected result: the terminal shows the active Cloudflare account. Do not publish screenshots of account details.

16. Choose installation names

Record the following values in a private note:

```
Development Worker name: example-rangkumin-development
Production Worker name: example-rangkumin
Development D1 name: example-rangkumin-development-db
Production D1 name: example-rangkumin-production-db
Application domain: finance.example.com
User 1 email: ...
User 2 email: ...
```

Use your own unique names. Do not reuse database IDs or domains from the official installation.

17. Create D1 databases

Run the first command with the development database name:

```
npx wrangler d1 create example-rangkumin-development-db
```

Run the second command with the production database name:

```
npx wrangler d1 create example-rangkumin-production-db
```

Each command displays a `database_name` and `database_id`.

1. Store both results in a private note.
2. Do not swap the development and production IDs.
3. Do not share screenshots containing account metadata.

18. Edit the Wrangler configuration

1. Open `wrangler.jsonc` using Visual Studio Code or a text editor.
2. At the top of the file, replace `name` with the development Worker name.
3. In the top-level `d1_databases`, replace the development `database_name` and `database_id`.
4. Find the `env.production` section.
5. Replace `env.production.name` with the production Worker name.
6. Replace the production `database_name` and `database_id`.
7. Replace the `rangkumin.dikadevit.my.id` route with your application domain.
8. Keep the binding names `DB`, `AI`, and `ASSETS`.
9. Keep `workers_dev: false` and `preview_urls: false` for production.
10. Save the file.

Check again that the development ID is not placed in the production section.

19. Create the production database structure

Run:

```
npm run db:migrations:list:production
npm run db:migrate:production
npm run db:migrations:list:production
```

If asked for confirmation, answer `yes`.

Expected result: the latest migration, including `0008_disable_telegram_channel.sql`, is successful.

Never run `npm run db:seed:local` or the development seed against production.

20. Add two production users

Windows PowerShell

```
Copy-Item "config/users.production.example.json" "config/users.production.local.json"
```

macOS/Linux

```
cp config/users.production.example.json config/users.production.local.json
```

Open `config/users.production.local.json`, then replace only the email addresses and display names:

```
{
  "users": [
    {
      "id": "user-1",
      "email": "USER_1_EMAIL",
      "displayName": "USER_1_NAME"
    },
    {
      "id": "user-2",
      "email": "USER_2_EMAIL",
      "displayName": "USER_2_NAME"
    }
  ]
}
```

Rules:

- use exactly two users;
- do not change the `user-1` and `user-2` IDs;
- both email addresses must be different;
- use the same addresses in Cloudflare Access;
- never commit the `.local.json` file.

Validate and save users to D1:

```
npm run db:provision:production
npm run db:provision:production -- --apply
```

Expected result: the terminal confirms that two production users were provisioned without printing their identities.

21. Create Cloudflare Access protection

1. Sign in at https://dash.cloudflare.com/.
2. Open **Zero Trust**.
3. If this is your first use, create a Zero Trust team name.
4. Open **Access controls** → **Applications**.
5. Click **Add an application**.
6. Select **Self-hosted**.
7. Enter an application name, such as `Rangkumin`.
8. Add the public hostname, such as `finance.example.com`.
9. Create a policy with the **Allow** action.
10. Select **Emails** as the rule selector.
11. Enter only the two production email addresses.
12. Choose an identity provider, such as **One-time PIN** or Google.
13. Save the application.
14. Open the application details.
15. Copy the **Application Audience (AUD) Tag**.
16. Record the team domain, such as `https://teamname.cloudflareaccess.com`.

Do not create an `Allow Everyone` policy. Rangkumin is designed for two users.

22. Prepare Cloudflare Access secrets

Copy the example config.

Windows PowerShell

```
Copy-Item "config/access.production.example.json" "config/access.production.local.json"
```

macOS/Linux

```
cp config/access.production.example.json config/access.production.local.json
```

Open the new file and set:

```
{
  "ACCESS_TEAM_DOMAIN": "https://teamname.cloudflareaccess.com",
  "ACCESS_AUD": "AUD_FROM_ACCESS_APPLICATION"
}
```

Validate without uploading secrets:

```
npm run access:secrets:production
```

If validation fails, make sure the domain uses HTTPS and the AUD was copied completely without spaces.

23. Create Web Push keys

Run once:

```
npx --yes web-push generate-vapid-keys --json
```

The command displays a public key and private key. Never screenshot or share the private key.

Copy the example config.

Windows PowerShell

```
Copy-Item "config/web-push.production.example.json" "config/web-push.production.local.json"
```

macOS/Linux

```
cp config/web-push.production.example.json config/web-push.production.local.json
```

Set the new file:

```
{
  "WEB_PUSH_VAPID_SUBJECT": "mailto:CONTACT_EMAIL",
  "WEB_PUSH_VAPID_PUBLIC_KEY": "PUBLIC_KEY_FROM_COMMAND",
  "WEB_PUSH_VAPID_PRIVATE_KEY": "PRIVATE_KEY_FROM_COMMAND"
}
```

Validate:

```
npm run webpush:secrets:production
```

Keep a backup of the private key in a password manager or secret manager. If the key is lost or changed, devices must subscribe again.

24. Enable the Receipt Scan model

1. Open Cloudflare Dashboard.
2. Open **Workers AI**.
3. Find `@cf/meta/llama-3.2-11b-vision-instruct`.

4. Open the model or AI Playground.
5. Accept the Meta license when requested.
6. Check the account pricing and limits.

No API token needs to be stored in source code. The `AI` binding is attached when the Worker is deployed.

25. Check the application before deployment

Run each command separately:

```
npm run format:check
npm run lint
npm run typecheck
npm test
npm run build
```

Do not continue if a command fails. Save a sanitized error message if you need assistance.

Expected result: formatting, lint, typecheck, 153 tests, and build all succeed.

26. Deploy the Worker

Run:

```
npx wrangler deploy --env production
```

Wait for Wrangler to display:

- a successful Worker upload;
- an active custom domain;
- the `*/15 * * * *` cron;
- the `15 17 * * *` cron;
- a new Version ID.

If the domain has a conflicting DNS record, remove or correct that record with the domain owner's approval, then deploy again.

27. Upload secrets to Cloudflare

After the Worker exists, run:

```
npm run access:secrets:production -- --apply
npm run webpush:secrets:production -- --apply
```

List secret names without displaying values:

```
npx wrangler secret list --env production
```

Expected result: these five secret names are present:

- `ACCESS_TEAM_DOMAIN`
- `ACCESS_AUD`
- `WEB_PUSH_VAPID_SUBJECT`
- `WEB_PUSH_VAPID_PUBLIC_KEY`
- `WEB_PUSH_VAPID_PRIVATE_KEY`

28. Test the production website

1. Open the application domain in an incognito/private window.
2. Confirm that Cloudflare Access appears.
3. Sign in with User 1's email address.
4. Confirm that the dashboard and correct user name appear.
5. Sign out or use another browser.
6. Sign in with User 2's email address.
7. Confirm that the dashboard and correct user name appear.
8. Try an email address that is not allowed; access must be denied.

29. Test the main features

Use dummy data and complete this checklist:

1. User 1 creates an income transaction.
2. User 1 creates an expense transaction.
3. User 2 can see User 1's transactions.
4. User 2 cannot edit or delete User 1's transactions.
5. Enable Web Push on both devices.
6. Create a transaction and confirm that the partner receives a notification.
7. Scan a receipt and confirm that the result remains a draft until approved.
8. Create a savings goal, then deposit, withdraw, and transfer.
9. Create a budget and reminder.
10. Export CSV and Excel files.
11. Import dummy data and review the preview before committing.
12. Install the PWA, open it online once, and test an offline reload.

On iPhone/iPad, add the application to the Home Screen and open it from that icon before enabling Web Push.

30. Check scheduled jobs

1. Open Cloudflare Dashboard.
2. Open **Workers & Pages**.
3. Select the production Worker.
4. Open **Triggers**.
5. Confirm that two cron schedules are present.

The 15-minute cron processes reminders, budgets, and Web Push. The daily cron runs normal processing and purges Trash.

31. Update the application later

Read release notes and back up data before an update. The general flow is:

```
git pull
npm ci
npm run db:migrations:list:production
npm run db:migrate:production
npm test
npm run build
npx wrangler deploy --env production
```

Never edit an old migration, and never apply a production migration without reviewing the new migration list.

32. Common troubleshooting

`node` or `npm` is not found

Install Node.js 22+, restart the terminal or computer, then check `node --version`.

`wrangler login` does not open a browser

Copy the login URL from the terminal into a browser. Make sure popups and the firewall are not blocking it.

Migration says the database does not exist

Check the `database_id` and `database_name` under the correct environment in `wrangler.jsonc`.

Access login succeeds but the application returns 403

The Cloudflare Access email address does not match the D1 user. Correct `users.production.local.json`, then run provisioning with `--apply` again.

Web Push does not arrive

- confirm that browser permission is allowed;
- confirm that all five secrets exist;
- disable and re-enable notifications from Settings;
- on iOS, open the application from the Home Screen;
- inspect Worker logs with `npx wrangler tail --env production`.

Receipt Scan returns 502

- confirm that the Meta license was accepted;
- check Workers AI quota and billing;
- use a clear JPEG/PNG/WebP image;
- inspect Worker logs without sharing the receipt image.

The website does not open after deployment

Check the custom domain, DNS conflicts, Cloudflare Access hostname, and latest deployment status.

`npm test` takes time to exit

Wait for the Worker tests to finish. Tests use a hermetic configuration and do not require a Cloudflare API token.

33. Files that must remain private

Never commit or send these files:

```
.dev.vars
config/users.production.local.json
config/access.production.local.json
config/web-push.production.local.json
```

Never store exports containing real transactions in the repository. `.gitignore` helps, but you must still inspect `git status` before every commit.

34. Getting help

- General bugs: open a GitHub Issue using dummy data.

- Vulnerabilities: use GitHub Security Advisories, not a public issue.
- Setup, support, custom branding, or managed services: see `COMMERCIAL.md`.
- Technical documentation: see `docs/setup.en.md`.

The Rangkumin core uses the MIT License. Installation packages, support, customization, and private extensions may be offered under a separate commercial agreement.